MANUAL TÉCNICO — AGROGESTIÓN

Índice

1. Introducción
2. Arquitectura de la aplicación
    - 2.1. Frontend
        - 2.1.1. Tecnologías usadas
        - 2.1.2. Entorno de desarrollo
    - 2.2. Backend
        - 2.2.1. Tecnologías usadas
        - 2.2.2. Entorno de desarrollo
3. Documentación técnica
    - 3.1. Análisis
        - 3.1.1. Requisitos funcionales
        - 3.1.2. Requisitos no funcionales
        - 3.1.3. Modelo de datos (Base de datos)
        - 3.1.4. Diagrama Entidad-Relación
        - 3.1.5. Diccionario de datos
        - 3.1.6. Casos de uso
    - 3.2. Desarrollo (Diagramas de secuencia)
    - 3.3. Pruebas realizadas
4. Proceso de despliegue
    - 4.1. Requisitos de sistema
    - 4.2. Configuración de Supabase
    - 4.3. Subida de la aplicación a producción
5. Propuestas de mejoras
    - 5.1. Mejoras aplicadas durante el desarrollo
    - 5.2. Mejoras propuestas para futuras versiones
6. Bibliografía

---

1. Introducción

Agrogestión es una aplicación web desarrollada como proyecto final del ciclo formativo de Grado Superior en Desarrollo de Aplicaciones Web (DAW) en el IES Albarregas de Mérida.

El objetivo del proyecto es crear una herramienta web que permita digitalizar la gestión del trabajo en explotaciones agrícolas, facilitando la coordinación entre los distintos roles que intervienen en la actividad: administradores, gerentes, capataces y trabajadores.

La aplicación se ha desarrollado siguiendo una arquitectura moderna de tipo **SPA (Single Page Application)** con un frontend en React y un backend gestionado íntegramente por Supabase (Backend as a Service).

Objetivos del proyecto

- Permitir el registro y autenticación de usuarios.

- Implementar un sistema de roles jerárquico (ADMIN, GERENTE, CAPATAZ, TRABAJADOR).
- Gestionar terrenos agrícolas (alta, baja lógica, edición).
- Crear, asignar y hacer seguimiento de tareas agrícolas.
- Establecer relaciones entre gerentes-capataces y capataces-trabajadores.
- Garantizar la seguridad de los datos mediante políticas RLS (Row Level Security).
- Ofrecer una interfaz responsive accesible desde cualquier dispositivo.

---

2. Arquitectura de la aplicación

Agrogestión utiliza una arquitectura **cliente-servidor desacoplada**:

```
┌──────────────────────┐                ┌──────────────────────────┐
│                      │   HTTPS/REST   │                          │
│   FRONTEND (SPA)     │ ◄────────────► │   BACKEND (Supabase)     │
│   React + Vite       │                │   PostgreSQL + Auth      │
│   TailwindCSS        │                │   RLS + API REST         │
│                      │                │                          │
│   Navegador web      │                │   Servidor en la nube    │
└──────────────────────┘                └──────────────────────────┘
```

- **Frontend:** Aplicación SPA que se ejecuta en el navegador del usuario.
- **Backend:** Supabase proporciona base de datos PostgreSQL, autenticación, API REST auto-generada y políticas de seguridad.
- **Comunicación:** El frontend se comunica con Supabase a través de la librería `@supabase/supabase-js` mediante llamadas HTTPS.

2.1. Frontend

2.1.1. Tecnologías usadas

Tecnología	Versión	Descripción
React	19.2.0	Librería para construir interfaces de usuario declarativas.
TypeScript	5.9.3	Superset de JavaScript con tipado estático.
Vite	7.2.4	Herramienta de build ultrarrápida para proyectos frontend.
TailwindCSS	4.1.18	Framework de utilidades CSS para diseño responsive.
@vitejs/plugin-react-swc	4.2.2	Plugin de Vite que usa SWC para compilar React (más rápido).
React Router	7.13.1	Librería de enrutamiento para navegación SPA.
i18next / react-i18next	25.8.14 / 16.5.4	Librería de internacionalización (i18n) para soporte multi-idioma.
Zustand	5.0.11	Gestión de estado global ligera y sencilla.

Tecnología	Versión	Descripción
dotenv	17.2.3	Gestión de variables de entorno.
ESLint	9.39.1	Herramienta de análisis estático para detectar errores en JS/TS.
typescript-eslint	8.46.4	Plugin de ESLint para soporte de TypeScript.

2.1.2. Entorno de desarrollo

Componente	Detalle
Sistema operativo	Windows 10/11
IDE	Visual Studio Code
Runtime	Node.js (versión LTS recomendada: 20.x o superior)
Gestor de paquetes	npm (incluido con Node.js)
Control de versiones	Git + GitHub (repositorio: Juanfrina/Agrogestion.Web)
Navegador de pruebas	Google Chrome / Firefox / Edge

2.2. Backend

2.2.1. Tecnologías usadas

Tecnología	Descripción
Supabase	Backend as a Service (BaaS) basado en PostgreSQL.
PostgreSQL	Motor de base de datos relacional (gestionado por Supabase).
Supabase Auth	Sistema de autenticación integrado (email + contraseña).
Supabase API REST	API auto-generada a partir del esquema de la base de datos.
Row Level Security (RLS)	Políticas de seguridad a nivel de fila en PostgreSQL.
@supabase/supabase-js	2.93.3 — Librería cliente para comunicar el frontend con Supabase.

2.2.2. Entorno de desarrollo

Componente	Detalle
Plataforma	Supabase Cloud (https://supabase.com)
Base de datos	PostgreSQL gestionado en la nube de Supabase
Panel de administración	Dashboard web de Supabase para configuración de tablas y RLS
Variables de entorno	VITE_SUPABASE_URL y VITE_SUPABASE_ANON_KEY en archivo .env

3. Documentación técnica

3.1. Análisis

3.1.1. Requisitos funcionales

ID	Requisito	Prioridad
RF-01	El sistema debe permitir el registro de nuevos usuarios mediante email y contraseña.	Alta
RF-02	El sistema debe permitir el inicio de sesión con autenticación segura.	Alta
RF-03	El sistema debe asignar un panel diferente a cada usuario según su rol.	Alta
RF-04	El administrador debe poder ver, crear, editar y desactivar usuarios.	Alta
RF-05	El administrador debe poder asignar roles a los usuarios.	Alta
RF-06	El gerente debe poder gestionar terrenos (alta, baja lógica, edición).	Alta
RF-07	El gerente debe poder crear tareas agrícolas asociadas a terrenos.	Alta
RF-08	El gerente debe poder asignar tareas a capataces.	Alta
RF-09	El gerente debe poder consultar el estado de las tareas.	Alta
RF-10	El gerente debe poder asociar y desasociar capataces a su equipo.	Media
RF-11	El capataz debe poder ver las tareas que le han sido asignadas.	Alta
RF-12	El capataz debe poder cambiar el estado de una tarea (pendiente → en progreso → completada).	Alta
RF-13	El capataz debe poder asignar trabajadores de apoyo a tareas concretas.	Media
RF-14	El capataz debe poder gestionar su equipo de trabajadores habituales.	Media
RF-15	El trabajador debe poder ver las tareas en las que participa.	Alta

ID	Requisito	Prioridad
RF-16	El trabajador debe poder consultar los detalles de cada tarea asignada.	Media
RF-17	El sistema debe aplicar filtros por terreno, estado y usuario en los listados.	Media
RF-18	El sistema debe incluir una landing page informativa con acceso a login y registro.	Baja
RF-19	El sistema debe permitir cerrar sesión de forma segura.	Alta
RF-20	Cada usuario solo debe poder acceder a los datos que le corresponden según su rol.	Alta

3.1.2. Requisitos no funcionales

ID	Requisito	Categoría
RNF-01	La aplicación debe ser responsive y funcionar en dispositivos móviles, tablets y PCs.	Usabilidad
RNF-02	La interfaz debe ser intuitiva y no requerir formación previa.	Usabilidad
RNF-03	El tiempo de carga inicial no debe superar los 3 segundos con conexión estándar.	Rendimiento
RNF-04	Las consultas a base de datos deben responder en menos de 1 segundo.	Rendimiento
RNF-05	La autenticación debe usar tokens JWT seguros gestionados por Supabase Auth.	Seguridad
RNF-06	Las políticas RLS deben garantizar que ningún usuario acceda a datos no autorizados.	Seguridad
RNF-07	Las contraseñas deben almacenarse cifradas (bcrypt) en Supabase Auth.	Seguridad
RNF-08	La comunicación con el servidor debe realizarse siempre sobre HTTPS.	Seguridad
RNF-09	El código fuente debe seguir convenciones de estilo validadas con ESLint.	Mantenibilidad
RNF-10	La aplicación debe usar TypeScript para minimizar errores en tiempo de desarrollo.	Mantenibilidad
RNF-11	La estructura del proyecto debe ser modular y escalable.	Mantenibilidad

ID	Requisito	Categoría
RNF-12	La aplicación debe ser compatible con Chrome, Firefox, Edge y Safari actualizados.	Compatibilidad
RNF-13	El sistema debe estar disponible 24/7 gracias al alojamiento en Supabase Cloud.	Disponibilidad

3.1.3. Modelo de datos (Base de datos)

La base de datos se implementa en PostgreSQL gestionado por Supabase. Se compone de 8 tablas principales interrelacionadas.

Tablas del sistema:

Tabla	Descripción
rol	Tipos de usuario: ADMIN, GERENTE, CAPATAZ, TRABAJADOR.
usuario	Perfil extendido del usuario (nombre, apellidos, rol, etc.).
terreno	Terrenos gestionados por los gerentes.
estados_tarea	Estados posibles de una tarea (pendiente, en progreso, completada).
tarea	Tareas agrícolas vinculadas a terrenos, gerentes y capataces.
gerente_capataz	Relación N:M entre gerentes y capataces.
tarea_trabajador	Trabajadores que ayudan a un capataz en una tarea concreta.
capataz_trabajador	Relación estable entre capataz y sus trabajadores habituales.

3.1.4. Diagrama Entidad-Relación

```
┌──────────┐        ┌──────────────┐        ┌──────────────────┐
│   ROL    │1──────N│   USUARIO    │        │   ESTADOS_TAREA  │
│          │        │              │        │                  │
│ id(PK)   │        │ id(PK)       │        │ id(PK)           │
│ nombre   │        │ nombre       │        │ nombre           │
│          │        │ apellidos    │        │ descripcion      │
└──────────┘        │ email        │        └──────────────────┘
                    │ id_rol(FK)   │                  │1
                    │ activo       │                  │
                    │ created_at   │                  │
                    └──────────────┘                  │
                           │                          │
                           │                          │
                  ┌────────┼────────┐                 │
                  │        │        │                 │
              ┌───▼────┐ ┌─▼──────┐ ┌▼──────────┐     │
              │GERENTE │ │CAPATAZ │ │TRABAJADOR │     │
              └────────┘ └────────┘ └───────────┘     │
                  │        │        │                 │
```

```
          │N         M│            │              │
     ┌────▼────────────▼────┐       │              │
     │  GERENTE_CAPATAZ     │       │              │
     │                      │       │              │
     │ id_gerente(FK)       │       │              │
     │ id_capataz(FK)       │       │              │
     └──────────────────────┘       │              │
          │                         │              │
          │1                        │              │
     ┌────▼──────────┐              │              │
     │   TERRENO     │              │              │
     │               │              │              │
     │ id(PK)        │              │              │
     │ nombre        │              │              │
     │ ubicacion     │              │              │
     │ superficie    │              │              │
     │id_gerente(FK)│              │              │
     │ activo        │              │              │
     └───────────────┘              │              │
          │1                        │              │
          │                         │              │
     ┌────▼──────────────────┐      │              │
     │       TAREA           │      │              │
     │                       │──────┘N             │
     │ id(PK)                │                     │
     │ descripcion           │─────────────────────┘N
     │ id_terreno(FK)        │
     │ id_gerente(FK)        │
     │ id_capataz(FK)        │
     │ id_estado(FK)         │
     │ fecha_creacion        │
     └───────────────────────┘
              │1
              │
     ┌────────▼──────────────┐
     │   TAREA_TRABAJADOR    │
     │                       │
     │ id_tarea(FK)          │
     │ id_trabajador(FK)     │
     └───────────────────────┘


     ┌───────────────────────┐
     │  CAPATAZ_TRABAJADOR   │
     │                       │
     │ id_capataz(FK)        │
     │ id_trabajador(FK)     │
     └───────────────────────┘
```

3.1.5. Diccionario de datos

Tabla: rol

Campo	Tipo	Restricciones	Descripción
id	SERIAL	PK, NOT NULL	Identificador único del rol.
nombre	VARCHAR(50)	NOT NULL, UNIQUE	Nombre del rol (ADMIN, GERENTE...).

Tabla: usuario

Campo	Tipo	Restricciones	Descripción
id	UUID	PK, NOT NULL	Identificador único (vinculado a Auth).
nombre	VARCHAR(100)	NOT NULL	Nombre del usuario.
apellidos	VARCHAR(150)	NOT NULL	Apellidos del usuario.
email	VARCHAR(255)	NOT NULL, UNIQUE	Correo electrónico.
id_rol	INTEGER	FK → rol(id)	Rol asignado al usuario.
activo	BOOLEAN	DEFAULT TRUE	Indica si la cuenta está activa.
created_at	TIMESTAMP	DEFAULT NOW()	Fecha de creación del registro.

Tabla: terreno

Campo	Tipo	Restricciones	Descripción
id	SERIAL	PK, NOT NULL	Identificador único del terreno.
nombre	VARCHAR(150)	NOT NULL	Nombre del terreno.
ubicacion	VARCHAR(255)		Dirección o referencia geográfica.
superficie	DECIMAL(10,2)		Superficie en hectáreas.
id_gerente	UUID	FK → usuario(id)	Gerente que gestiona el terreno.
activo	BOOLEAN	DEFAULT TRUE	Estado activo/inactivo (baja lógica).
created_at	TIMESTAMP	DEFAULT NOW()	Fecha de creación.

Tabla: estados_tarea

Campo	Tipo	Restricciones	Descripción
id	SERIAL	PK, NOT NULL	Identificador del estado.
nombre	VARCHAR(50)	NOT NULL, UNIQUE	Nombre: pendiente, en progreso, completada.
descripcion	TEXT		Descripción del estado.

Tabla: tarea

Campo	Tipo	Restricciones	Descripción
id	SERIAL	PK, NOT NULL	Identificador de la tarea.

Campo	Tipo	Restricciones	Descripción
descripcion	TEXT	NOT NULL	Descripción de la tarea a realizar.
id_terreno	INTEGER	FK → terreno(id)	Terreno donde se realiza.
id_gerente	UUID	FK → usuario(id)	Gerente que creó la tarea.
id_capataz	UUID	FK → usuario(id)	Capataz asignado a ejecutar la tarea.
id_estado	INTEGER	FK → estados_tarea(id)	Estado actual de la tarea.
fecha_creacion	TIMESTAMP	DEFAULT NOW()	Fecha de creación.

Tabla: gerente_capataz

Campo	Tipo	Restricciones	Descripción
id_gerente	UUID	FK → usuario(id), PK	Gerente de la relación.
id_capataz	UUID	FK → usuario(id), PK	Capataz asociado al gerente.

Tabla: tarea_trabajador

Campo	Tipo	Restricciones	Descripción
id_tarea	INTEGER	FK → tarea(id), PK	Tarea en la que participa.
id_trabajador	UUID	FK → usuario(id), PK	Trabajador asignado como apoyo.

Tabla: capataz_trabajador

Campo	Tipo	Restricciones	Descripción
id_capataz	UUID	FK → usuario(id), PK	Capataz de la relación.
id_trabajador	UUID	FK → usuario(id), PK	Trabajador habitual del capataz.

3.1.6. Casos de uso

Actores del sistema:

Actor	Descripción
Visitante	Usuario no autenticado que accede a la landing page.
Admin	Administrador del sistema con control total.
Gerente	Gestiona terrenos, crea tareas y supervisa capataces.
Capataz	Ejecuta tareas, actualiza estados y coordina trabajadores.
Trabajador	Consulta las tareas en las que participa.

Casos de uso principales:

CU	Nombre	Actor(es)	Descripción
CU-01	Registrarse	Visitante	El visitante crea una cuenta con email y contraseña.
CU-02	Iniciar sesión	Todos	El usuario se autentica con email y contraseña.
CU-03	Cerrar sesión	Todos	El usuario cierra su sesión de forma segura.
CU-04	Ver panel según rol	Todos (autenticados)	El sistema redirige al panel correspondiente al rol.
CU-05	Gestionar usuarios	Admin	CRUD de usuarios y asignación de roles.
CU-06	Alta de terreno	Gerente	El gerente crea un nuevo terreno.
CU-07	Editar terreno	Gerente	El gerente modifica datos de un terreno.
CU-08	Baja lógica de terreno	Gerente	El gerente desactiva un terreno.
CU-09	Crear tarea	Gerente	El gerente crea una tarea asociada a un terreno.
CU-10	Asignar tarea a capataz	Gerente	El gerente asigna una tarea pendiente a un capataz.
CU-11	Ver estado de tareas	Gerente	El gerente consulta el estado de las tareas.
CU-12	Asociar capataz	Gerente	El gerente asocia un capataz a su equipo.
CU-13	Tomar tarea	Capataz	El capataz cambia una tarea de pendiente a en progreso.
CU-14	Completar tarea	Capataz	El capataz marca una tarea como completada.
CU-15	Asignar trabajador a tarea	Capataz	El capataz asigna un trabajador de apoyo a una tarea.
CU-16	Gestionar trabajadores	Capataz	El capataz administra su equipo de trabajadores habituales.
CU-17	Ver mis tareas asignadas	Trabajador	El trabajador consulta las tareas en las que participa.
CU-18	Filtrar tareas	Gerente, Capataz	Filtrado de tareas por terreno, estado o usuario.

Diagrama de casos de uso (representación textual):

```
                    SISTEMA AGROGESTIÓN
┌──────────────────────────────────────────────────┐
│                                                  │
│  ┌──────────────┐      ┌──────────────────────┐  │
│  │  Registrarse │      │  Iniciar sesión      │  │
│  └──────────────┘      └──────────────────────┘  │
│         │                       │                │
│  ┌──────▼───────────────────────▼──────────────┐ │
│  │         Ver panel según rol                 │ │
│  └─────────────────────────────────────────────┘ │
│      │          │          │          │          │
│  ┌───▼────┐ ┌───▼────┐ ┌───▼────┐ ┌──▼─────────┐ │
│  │ Admin  │ │Gerente │ │Capataz │ │Trabajador  │ │
│  │ Panel  │ │ Panel  │ │ Panel  │ │  Panel     │ │
│  └────────┘ └────────┘ └────────┘ └────────────┘ │
│      │          │          │          │          │
│  Gestión    Terrenos   Tomar     Ver mis         │
│  usuarios   Tareas     tareas    tareas          │
│  y roles    Capataces  Completar                 │
│                        Asignar                   │
│                        trabajadores              │
│                                                  │
│  ┌──────────────────────────────────────────┐    │
│  │           Cerrar sesión                  │    │
│  └──────────────────────────────────────────┘    │
└──────────────────────────────────────────────────┘
```

3.2. Desarrollo (Diagramas de secuencia)

3.2.1. Diagrama de secuencia: Inicio de sesión

```
Usuario          Frontend (React)        Supabase Auth         Base de datos
  │                    │                       │                      │
  │  Introduce email   │                       │                      │
  │  y contraseña      │                       │                      │
  │──────────────────►│                       │                      │
  │                    │                       │                      │
  │                    │  signInWithPassword() │                      │
  │                    │──────────────────────►│                      │
  │                    │                       │                      │
  │                    │                       │ Verifica credenciales│
  │                    │                       │──────────────────────►│
  │                    │                       │                      │
  │                    │                       │  ◄── Usuario válido  │
  │                    │                       │                      │
  │                    │ ◄── Token JWT + datos │                      │
  │                    │                       │                      │
  │                    │  Consulta rol usuario │                      │
```

```
    |                     |──────────────────────────────────────────────►|
    |                     |                         |                     |
    |                     |  ◄── Datos de rol       |                     |
    |                     |                         |                     |
    |  Redirige al panel  |                         |                     |
    |  según rol          |                         |                     |
    |◄────────────────────|                         |                     |
```

3.2.2. Diagrama de secuencia: Registro de usuario

```
Visitante        Frontend (React)        Supabase Auth         Base de datos
   |                     |                         |                     |
   |  Rellena formulario|                         |                     |
   |  de registro        |                         |                     |
   |────────────────────►|                         |                     |
   |                     |                         |                     |
   |                     |  signUp(email, pass)    |                     |
   |                     |────────────────────────►|                     |
   |                     |                         |                     |
   |                     |                         |  Crea usuario en Auth|
   |                     |                         |────────────────────►|
   |                     |                         |                     |
   |                     |  ◄── Usuario creado     |                     |
   |                     |                         |                     |
   |                     |  INSERT en tabla        |                     |
   |                     |  usuario (perfil)       |                     |
   |                     |──────────────────────────────────────────────►|
   |                     |                         |                     |
   |                     |  ◄── Perfil creado      |                     |
   |                     |                         |                     |
   |  Mensaje de éxito   |                         |                     |
   |◄────────────────────|                         |                     |
```

3.2.3. Diagrama de secuencia: Crear tarea (Gerente)

```
Gerente          Frontend (React)          Supabase API          Base de datos
   |                     |                         |                     |
   |  Rellena formulario|                         |                     |
   |  de nueva tarea     |                         |                     |
   |────────────────────►|                         |                     |
   |                     |                         |                     |
   |                     |  Valida datos locales   |                     |
   |                     |                         |                     |
   |                     |  INSERT tarea           |                     |
   |                     |  (terreno, capataz,     |                     |
   |                     |   estado: pendiente)    |                     |
   |                     |────────────────────────►|                     |
   |                     |                         |                     |
```

```
    |                    |                        |  Verifica RLS        |
    |                    |                        |  (es gerente?)       |
    |                    |                        |──────────────────────►|
    |                    |                        |                      |
    |                    |                        |  ◄── INSERT OK       |
    |                    |                        |                      |
    |                    |  ◄── Tarea creada      |                      |
    |                    |                        |                      |
    |  Actualiza listado |                        |                      |
    |  de tareas         |                        |                      |
    |◄───────────────────|                        |                      |
```

3.2.4. Diagrama de secuencia: Cambiar estado de tarea (Capataz)

```
Capataz          Frontend (React)          Supabase API          Base de datos
   |                    |                        |                      |
   |  Pulsa "Tomar      |                        |                      |
   |  tarea" o          |                        |                      |
   |  "Completar"       |                        |                      |
   |───────────────────►|                        |                      |
   |                    |                        |                      |
   |                    |  UPDATE tarea          |                      |
   |                    |  SET id_estado = X     |                      |
   |                    |───────────────────────►|                      |
   |                    |                        |                      |
   |                    |                        |  Verifica RLS        |
   |                    |                        |  (es capataz de      |
   |                    |                        |   esta tarea?)       |
   |                    |                        |──────────────────────►|
   |                    |                        |                      |
   |                    |                        |  ◄── UPDATE OK       |
   |                    |                        |                      |
   |                    |  ◄── Estado actualizado|                      |
   |                    |                        |                      |
   |  Actualiza vista   |                        |                      |
   |◄───────────────────|                        |                      |
```

3.3. Pruebas realizadas

3.3.1. Pruebas funcionales

ID	Prueba	Resultado esperado	Estado
PF-01	Registro con datos válidos	Se crea el usuario y se muestra mensaje de éxito.	☑ OK
PF-02	Registro con email ya existente	Se muestra error indicando que el email ya existe.	☑ OK

ID	Prueba	Resultado esperado	Estado
PF-03	Login con credenciales correctas	Se redirige al panel correspondiente al rol.	☑ OK
PF-04	Login con credenciales incorrectas	Se muestra mensaje de error.	☑ OK
PF-05	Cerrar sesión	Se cierra la sesión y se redirige a la landing.	☑ OK
PF-06	Admin: ver lista de usuarios	Se muestra la tabla con todos los usuarios.	☑ OK
PF-07	Admin: asignar rol a usuario	El rol se actualiza en la base de datos.	☑ OK
PF-08	Gerente: crear terreno	El terreno aparece en el listado del gerente.	☑ OK
PF-09	Gerente: baja lógica de terreno	El terreno se marca como inactivo.	☑ OK
PF-10	Gerente: crear tarea	La tarea se asocia al terreno y capataz.	☑ OK
PF-11	Capataz: tomar tarea	El estado cambia a "en progreso".	☑ OK
PF-12	Capataz: completar tarea	El estado cambia a "completada".	☑ OK
PF-13	Capataz: asignar trabajador a tarea	El trabajador aparece asociado a la tarea.	☑ OK
PF-14	Trabajador: ver tareas asignadas	Se muestran solo las tareas del trabajador.	☑ OK
PF-15	Acceso no autorizado a datos de otro rol	Supabase RLS bloquea el acceso.	☑ OK

3.3.2. Pruebas de interfaz / Responsive

ID	Prueba	Navegador/Dispositivo	Estado
PI-01	Visualización en escritorio (1920x1080)	Chrome	☑ OK
PI-02	Visualización en tablet (768x1024)	Firefox	☑ OK
PI-03	Visualización en móvil (375x667)	Chrome (DevTools)	☑ OK
PI-04	Navegación con menú hamburguesa en móvil	Chrome (DevTools)	☑ OK
PI-05	Formularios responsive	Edge	☑ OK

3.3.3. Pruebas de seguridad

ID	Prueba	Resultado esperado	Estado
PS-01	Acceso directo a URL protegida sin sesión	Redirige al login.	☑ OK
PS-02	Inyección SQL vía formularios	Supabase sanitiza las consultas automáticamente.	☑ OK
PS-03	Acceso a datos de otro usuario vía API	RLS bloquea el acceso.	☑ OK

4. Proceso de despliegue

4.1. Requisitos de sistema

Para desarrollo local:

Requisito	Versión mínima
Node.js	18.x o superior (recomendado 20.x LTS)
npm	9.x o superior (incluido con Node.js)
Git	2.x o superior
Navegador web	Chrome 90+, Firefox 90+, Edge 90+, Safari 15+
Sistema operativo	Windows 10+, macOS 12+, Linux (Ubuntu 20.04+)

Para producción:

Requisito	Detalle
Cuenta en Supabase	Plan gratuito o de pago.
Plataforma de hosting	Vercel, Netlify u otro hosting estático.
Dominio (opcional)	Dominio personalizado si se desea.

4.2. Configuración de Supabase

1. Acceder a https://supabase.com y crear una cuenta.
2. Crear un nuevo proyecto con nombre "agrogestion".
3. En el panel de Supabase, acceder al **SQL Editor** y ejecutar los scripts de creación de tablas.
4. Configurar las **políticas RLS** para cada tabla según el rol del usuario.
5. En **Settings** → **API**, copiar la `URL del proyecto` y la `anon key`.
6. Crear el archivo `.env` en la raíz del proyecto frontend con:

```
VITE_SUPABASE_URL=https://tu-proyecto.supabase.co
VITE_SUPABASE_ANON_KEY=tu_clave_anon_publica
```

4.3. Subida de la aplicación a producción

Opción A: Despliegue en Vercel

1. Crear una cuenta en https://vercel.com.
2. Conectar el repositorio de GitHub (Juanfrina/Agrogestion.Web).
3. Configurar las variables de entorno en el panel de Vercel:
   - `VITE_SUPABASE_URL`
   - `VITE_SUPABASE_ANON_KEY`
4. Vercel detectará automáticamente que es un proyecto Vite y lo construirá.
5. La URL de producción estará disponible tras el despliegue.

Opción B: Despliegue en Netlify

1. Crear una cuenta en https://netlify.com.
2. Importar el repositorio de GitHub.
3. Configurar:
   - **Build command:** `npm run build`
   - **Publish directory:** `dist`
4. Añadir variables de entorno.
5. Publicar.

Opción C: Despliegue manual (servidor propio)

1. Ejecutar `npm run build` para generar la carpeta `dist/`.
2. Subir el contenido de `dist/` al servidor web (Apache, Nginx, etc.).
3. Configurar el servidor para servir una SPA (redirigir todas las rutas a `index.html`).

---

5. Propuestas de mejoras

5.1. Mejoras aplicadas durante el desarrollo

Mejora	Descripción
Migración a TypeScript	Se migró de JavaScript a TypeScript para mayor seguridad de tipos.
Uso de SWC en lugar de Babel	Compilación más rápida en desarrollo usando el plugin SWC.
Estructura modular del proyecto	Separación en carpetas: pages, components, context, hooks, router, database, interfaces, store, utils, lib.
TailwindCSS v4	Uso de la última versión de Tailwind con mejor rendimiento.
Variables de entorno con Vite	Uso de `import.meta.env` para gestionar variables sensibles.
Patrón Repository	Capa de abstracción para acceso a datos con implementaciones Supabase.

Mejora	Descripción
Internacionalización (i18n)	Soporte multi-idioma (español e inglés) con i18next.
Recuperación de contraseña	Flujo de restablecimiento de contraseña con Supabase Auth.
Store de estado global (Zustand)	Gestión centralizada del estado de autenticación con Zustand.
Gráficos mensuales	Componente `MensualChart` para visualización de datos con gráficos de barras mensuales.

5.2. Mejoras propuestas para futuras versiones

Mejora	Descripción
Notificaciones en tiempo real	Usar Supabase Realtime para notificar cambios de estado en tareas.
Exportación de informes en PDF	Generar informes de tareas y terrenos en formato PDF.
Geolocalización de terrenos	Integrar un mapa interactivo (Leaflet/Google Maps) para ubicar terrenos.
Sistema de comentarios en tareas	Permitir que capataces y trabajadores añadan comentarios.
Dashboard con estadísticas	Ampliar el panel de gráficos con más métricas y filtros avanzados.
Modo offline con Service Worker	Cachear datos para poder consultar información sin conexión.
Tests automatizados	Implementar testing con Vitest y React Testing Library.
Carga de imágenes	Subir fotos de terrenos o tareas usando Supabase Storage.
App móvil nativa	Versión con React Native o PWA para móviles.

6. Bibliografía

Recurso	URL
React – Documentación oficial	https://react.dev
Vite – Documentación oficial	https://vite.dev
TypeScript – Documentación oficial	https://www.typescriptlang.org/docs/
TailwindCSS – Documentación oficial	https://tailwindcss.com/docs
Supabase – Documentación oficial	https://supabase.com/docs
Supabase Auth – Guía de autenticación	https://supabase.com/docs/guides/auth

Recurso	URL
Supabase RLS – Row Level Security	https://supabase.com/docs/guides/auth/row-level-security
@supabase/supabase-js – Librería cliente	https://github.com/supabase/supabase-js
React Router – Documentación	https://reactrouter.com
ESLint – Documentación oficial	https://eslint.org
Node.js – Documentación oficial	https://nodejs.org
npm – Documentación oficial	https://docs.npmjs.com
MDN Web Docs – Referencia web	https://developer.mozilla.org
Vercel – Plataforma de despliegue	https://vercel.com/docs
Netlify – Plataforma de despliegue	https://docs.netlify.com

Agrogestión — Manual Técnico Autor: Juan Francisco Hurtado Pérez Ciclo: CFGS Desarrollo de Aplicaciones Web (DAW) Centro: IES Albarregas – Mérida (España)